Pierce Brosnan, Agent of Automation: Riding the Tsunami with LLMs

Joscha Lasse Bisping (401424), Debug Dingos

ABSTRACT

Can a large language model autonomously write, debug, and deliver real security scanner plugins? In this project, we put this question to the test by developing an agent that generates plugins for the Tsunami Security Scanner, targeting the OWASP Juice Shop as a benchmark. Our LangChain-powered agent not only creates Java-based plugins but also automatically compiles and refines them using a feedback-driven loop. This eliminates the need for manual coding or debugging. In controlled evaluation, the generated plugins successfully detected 16 distinct vulnerabilities, dramatically outpacing traditional manual development in both speed and reliability. We analyze the capabilities and limitations of this approach, explore its potential for broader security automation, and discuss how future iterations could bridge the gap between classroom challenges and real-world security engineering.

1 INTRODUCTION

Tsunami is a general-purpose network security scanner developed by Google, designed with an extensible plugin system for detecting high-severity vulnerabilities with high confidence [3]. By adding **vulnerability detection plugins**, Tsunami's capabilities can be expanded to identify new security issues in target systems. Google has even incentivized the community to contribute plugins through a patch reward program, underlining the importance of rapidly developing new detectors [5]. Traditionally, writing a Tsunami plugin is a manual task requiring developers to implement Java classes that conform to the scanner's framework, including writing a detector class with a `detect()` method and associated bootstrap code. This process can be time-consuming and repetitive, especially for straightforward vulnerabilities. Our project addresses this challenge by using a Large Language Model (LLM) agent to **automate the generation of Tsunami plugins**, reducing human effort in coding these detectors.

The motivation for this work stems from the observation that many vulnerability checks follow common patterns. For instance, a Tsunami plugin typically iterates over network services, applies a condition to check if each service is vulnerable, and then builds a report with details of the finding. Writing such boilerplate for each new vulnerability and often using similar logic with different payloads or endpoints is a tedious task that we hypothesize can be streamlined with AI. Recent advances in code generation, such as OpenAI's Codex [6] and Google's Gemini CLI [4], have shown that they can produce syntactically correct and logical code from natural language descriptions of tasks. Furthermore, agentic techniques allow an LLM not only to generate code but also to act on environment feedback like compiler errors or test results to iteratively refine its output. In this project, we leverage these capabilities to build a two-phase LLM workflow that writes, compiles, and self-corrects Tsunami security scanner plugins with minimal human intervention.

We target vulnerabilities in the OWASP Juice Shop, an intentionally vulnerable web application designed for training and testing security tools. The Juice Shop contains dozens of diverse vulnerabilities, making it a suitable testbed for our automated plugin generator. By providing the LLM agent with descriptions of Juice Shop vulnerabilities and examples of existing Tsunami plugins, we enable it to produce new detector plugins aimed at those vulnerabilities. Our results show that this approach can successfully generate working plugins. In our evaluation, the agent created 16 distinct vulnerability detection plugins that compiled and effectively detected the intended issues in Juice Shop, all without any manual coding.

2 BACKGROUND: TSUNAMI SCANNER, PLUGIN DEVELOPMENT AND OWASP JUICE SHOP

The Tsunami Security Scanner conducts scans in two main phases: (1) Reconnaissance, where it finds open ports and fingerprints services, and (2) Vulnerability Verification, where it runs applicable vulnerability detection plugins against the identified services. The plugin architecture allows Tsunami to invoke only the relevant checks based on the target's services (e.g. only run WordPress-related plugins if a WordPress service is detected). This design enables scalability, as new vulnerabilities can be supported simply by adding new plugins without altering the core scanner.

There are several ways to write a Tsunami vulnerability detection plugin. The preferred method is using a templated configuration format, which allows authors to specify certain checks in a high-level declarative manner. This covers many common vulnerability patterns without coding. However, not all vulnerabilities fit the template model. For more complex detection logic, Tsunami supports writing plugins in code in Java, but also Python or other languages via external scripts. In fact, Tsunami's documentation notes that if a vulnerability cannot be handled by the template format, developers can resort to a Java or Python plugin implementation. Java is used for first-class integration, since Tsunami itself is written in Java, whereas Python detectors run as separate processes invoked by Tsunami's tool interface. In the project, we focused on Java-based plugins, both because Java plugins are the default in Tsunami's examples and because they were given as example implementation to solving the Tsunami Challenge.

The structure of a Tsunami Java Plugin is essentially a class implementing the `VulnDetector` interface, annotated with `@PluginInfo` metadata. The metadata includes information like the plugin name, version, description, author, and what type of plugin it is. The core of the plugin is the implementation of the `detect( ... )` method, which Tsunami calls during scanning. Inside `detect`, the plugin inspects the target services and decides if each is vulnerable. A common pattern is to iterate over the list of candidate services, filter to the relevant service type (e.g. web services for a web vulnerability), and then call a helper function like `isServiceVulnerable(service)`

for the specific check. If a service is found vulnerable, the plugin returns a `DetectionReport` describing the vulnerability. For example, an HTTP vulnerability detector might check if a certain endpoint or URL on the service is accessible or returns an indicative response. Writing such a plugin manually involves knowing the details of the vulnerability, such as the URL path or payload that triggers it, and coding the logic to attempt the exploit and evaluate the response.

To evaluate our LLM agent, we needed a set of known vulnerabilities to generate plugins for. For this we were given the OWASP Juice Shop, an intentionally vulnerable web application that contains a wide range of common vulnerabilities, covering the OWASP Top Ten among others [8]. It includes challenges from common issues like SQL Injection and Cross-Site Scripting (XSS) to more complex logic flaws and CTF-style puzzles. In the context of Tsunami, the Juice Shop serves as a rich source of potential detection plugins: each known vulnerability could correspond to a plugin that checks for that specific weakness on a target web application. For our project, we focused on automating plugins for a subset of Juice Shop challenges. We provided the LLM agent with the descriptions and hints for these vulnerabilities, which we extracted from the Juice Shop documentation/challenges [7] and tasked it with generating plugins that would detect them.

This scenario simulates a real-world use-case: given a description of a new vulnerability or exploit technique, have an AI agent write a scanner plugin to detect it. Success is measured by whether the generated plugin can identify the vulnerability on the Juice Shop instance, earning a "point" for each detected issue in the classroom competition setup.

In total, our agent produced 16 working plugins, demonstrating a breadth of vulnerability coverage. While we do not detail each plugin here, they covered various categories from injection flaws, broken access controls, and sensitive data exposure analogous to the Juice Shop's range of challenges.

3 APPROACH AND SYSTEM DESIGN

Our solution evolved through several iterations. The key goal was to create a fully autonomous LLM-based workflow that takes a vulnerability description and outputs a complete, compilable/compiled Tsunami plugin for that vulnerability. We started with simpler approaches and gradually increased the agent's sophistication to overcome challenges like formatting errors and code correctness. Figure 1 illustrates our final system architecture, consisting of two main agent components working in sequence starting with generation and then debugging.

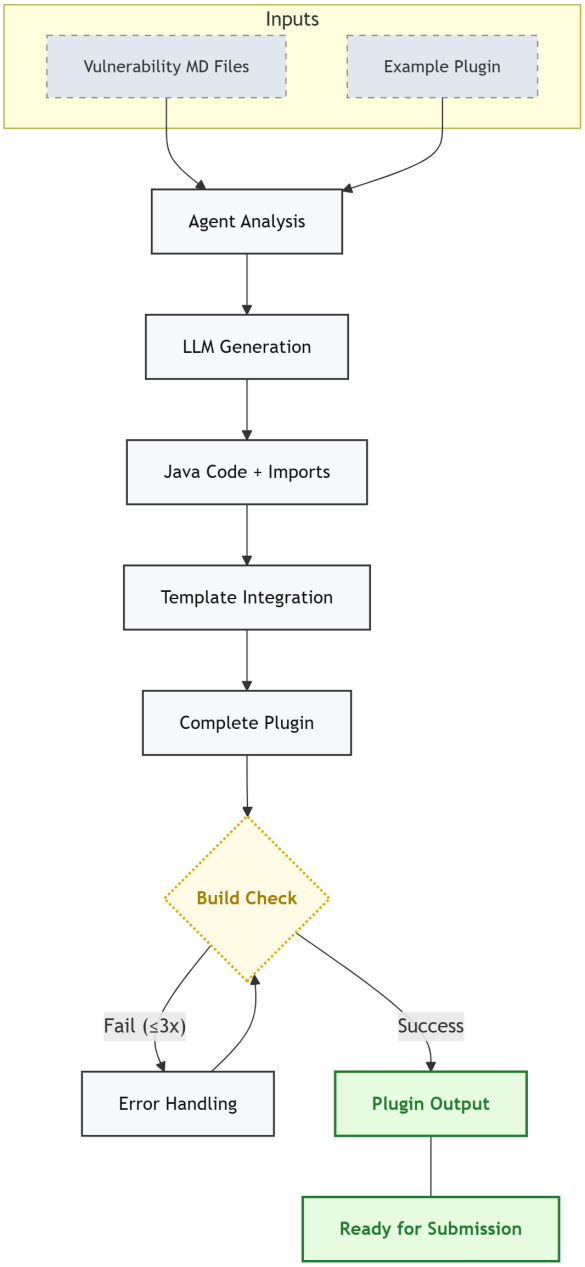

Figure 1: Final system architecture: Autonomous agent workflow to generate and verify a Tsunami plugin from a vulnerability description.

3.1 Initial Attempts: Template-Filling and Structured Output

Solution 1 – Manual Template Filling: The first approach was straightforward: we created a static Java plugin template with placeholders for elements like vulnerability name, description, and the core logic in `isServiceVulnerable`. We then prompted the LLM with a fixed instruction to generate the content for these placeholders given a vulnerability description. For example, we would prompt the model with *"Given the provided template, provide the Java code logic in the isServiceVulnerable method to detect X vulnerability. Additionally provide a short description and recommendation."* The LLM's output was then programmatically inserted into the template. This yielded a complete Java source file for the plugin, which we could compile. In some cases this worked, but often the inserted code had syntax issues, such as missing or wrong imports or braces. Since our approach at that time did not allow the model

to see compilation errors, these issues had to be fixed manually. This partially automated the process but did not eliminate human intervention.

Solution 2 – Structured JSON Output: To reduce syntax errors and better control the content, we attempted to have the LLM produce a structured JSON response containing all parts of the plugin. The idea was to make the output highly structured with keys for `imports`, `java_code` snippet, `plugin_name`, etc., so that our program could reliably parse it and place each piece into the corresponding spot in the template. The idea was taken from several online tutorials for LLM Agents, that mainly work with structured output. For that, we used a pydantic model, which got translated to JSON and told the agent to provide matching output:

```
class PluginImplementation(BaseModel):
    vulnerability_type: str
    plugin_name: str
    description: str
    recommendation: str
    endpoints: list[str]
    payloads: list[str]
    imports: list[str]
    java_code: str
```

In theory, this would let us easily extract the code and text fields. However, in practice this approach ran into significant difficulties. The main issue was that the model had to write the Java code as a JSON value and it had to do so in a very specific format so that the overall JSON structure would not be broken. In reality, this meant that even small mistakes such as an extra bracket, a missing quote, or unescaped special characters could corrupt the JSON and cause the parsing to fail. To deal with this, we had to implement numerous regex operations and data cleaning steps just to extract the code and metadata from the LLM's output.

This "cleaning and extraction" step quickly became a major source of complexity and error. For example, the model might return code with embedded explanations or comments that broke the JSON structure, or would occasionally forget to escape newlines and quotes, rendering the entire output unparsable. As a result, the workflow was fragile: the model would generate output, we would try to parse it, then have to clean or reconstruct it, and finally inject it into the plugin template. Any errors in this chain would require manual debugging.

Most importantly, this approach made it impossible to allow the model to self-correct across iterations. Because our data cleaning and extraction pipeline heavily modified the model's raw output before it was inserted into the plugin, there was no simple way for the model to "see" or reason about the exact code that actually ended up in the plugin source file. This disconnect prevented us from implementing an iterative, self-correcting workflow. When the initial output had issues, the model's subsequent attempts would often diverge further, since it was not operating on the real, cleaned code. For these reasons, the structured JSON approach ultimately proved unreliable for our use case.

3.2 Final Workflow: LLM Agent with Self-Correction

Our final design abandoned the strict JSON structure in favor of a more flexible, yet parseable, format and introduced an automated feedback loop. We built a LangChain-based agent [2] that operates in two phases: **Phase 1: Plugin Generation.** In this phase, the agent is given the context of an example plugin implementation as a model for structure and style and the specific vulnerability description which we derived from the Juice Shop docs [7]. We engineered the prompt to instruct the model to output two clearly separated sections: one for import statements and one for the core plugin code. For example, the agent's response would start with an `IMPORTS:` section listing any needed imports, followed by a Java code block containing the `isServiceVulnerable` method implementation and any other code necessary. By adopting this more natural, text-based output structure, aligned with how large language models are typically trained to present code, we reduced the frequency of formatting errors and improved both the reliability of parsing and the model's ability to follow instructions. An example output snippet from the agent for a generic vulnerability could look like this:

```
IMPORTS:
import com.google.protobuf.ByteString;
import java.net.URLEncoder;

```java
private boolean
    isServiceVulnerable(NetworkService
    networkService) {
    // Implement vulnerability check logic
        (e.g., send payload, observe
        response)
    try {
        String targetUrl =
            networkService.getHostname() +
            "/path/to/vulnerable/feature";
        // ... send HTTP request or perform
            check ...
        return true;  // or false based on
            response
    } catch (Exception e) {
        logger.atWarning().withCause(e).log("Error␣
            during␣vulnerability␣check");
        return false;
    }
}
```
```

Phase 2: Compilation & Self-Correction. Once the agent produces the plugin code, the next step is to ensure it compiles and works. We automated this by having the next instance of the agent invoke a `build-plugin` tool, given by the Raid Team, which was a wrapper for the typical Tsunami plugin compilation process. The

command essentially ran the Gradle build for the plugin and captured any compiler output. This output is fed back into the LLM if any error occurred. We set up a separate LangChain agent for this debugging phase, with a prompt designed to instruct the model to analyze compiler errors and suggest code fixes. Crucially, this second agent is not given the full context of the vulnerability again in order to avoid it diverging or changing the logic unnecessarily. It only sees the code and the error messages. For example, if the compiler output said "error: cannot find symbol: class URLEncoder", the agent would infer that an import for java.net.URLEncoder was missing or mistyped and add it to the imports section. Or, if the error was a syntax issue like a missing parenthesis, the agent would correct that in the code. The debugging agent would then output a revised code snippet in the same format as above including the IMPORT and code block. We would replace the previous code with the revised code and re-run the compilation. This loop continues until the code compiles successfully. In our experiments, we never needed more than one iteration in this loop. About 50% of the initial generations had some compilation error, but the agent always managed to fix the code in the very first fix attempt. The types of errors corrected included missing imports, minor syntax issues, or misused Tsunami API calls, all of which the LLM could recognize from the error messages and correct. This capability highlights the power of letting the LLM act as a self-debugging agent.

Internally, the implementation used Anthropic's Claude model (`claude-sonnet-4-20250514`) [1] via the LangChain [2] framework. Claude was chosen for its long context window and strong performance in code tasks. The agent prompt was carefully constructed to encourage step-by-step reasoning, which was especially important in the debugging phase. We essentially employed a "chain-of-thought" style prompt where the agent was instructed to consider the meaning of the error message and how to fix it before outputting a revised code version. Importantly, the agent itself does not directly execute the build-plugin command. Instead, our workflow triggers the build process externally and captures the compiler output. If the build fails, the agent is invoked with the error message and code context, prompting it to generate a corrected version. This separation simplifies automation and maintains a clear feedback loop between code generation, build execution, and error-driven refinement.

By the end of Phase 2, we obtain a compiled `.jar` plugin. All the above steps, from code generation to compilation and error correction, happen without direct human intervention in the process itself. The only required user input is the provision of the vulnerability context, that is, a description of how to find or exploit the vulnerability. For example, the user provides relevant details or documentation, such as the Juice Shop challenge description, and then triggers the agent with the vulnerability name. We created a script, `test_full_integration.sh`, that iterates through a list of vulnerability challenges and invokes the agent for each one. The agent then autonomously generates and refines each plugin based on the supplied context. The resulting plugins were subsequently submitted to the Tsunami Challenge for evaluation.

4 EVALUATION

We evaluated our LLM-generated plugins by submitting them to a controlled test environment in which Tsunami was executed against an instance of the OWASP Juice Shop. In this setup, Tsunami ran with all the generated plugins enabled, systematically scanning for vulnerabilities as defined in the provided vulnerability descriptions. Overall, 16 vulnerabilities were successfully detected by the generated plugins.

Each plugin was designed to cover an entire vulnerability category, often implementing multiple specific attack strategies or payloads targeting several related vulnerabilities within that type. Out of the total set of plugins generated, half compiled successfully on the first attempt, while the remaining half required a single round of automated corrections before compiling. None required more than two attempts, indicating the effectiveness of the error-driven refinement loop (see Table 1). The entire process from providing a vulnerability description to producing a compiled, tested plugin was highly automated. Human involvement was limited to supplying the initial context. Compared to the hours a manual implementation might require, this approach reduced plugin development and deployment to just a few minutes per vulnerability category.

Table 1: Effectiveness of LLM agent-based plugin generation and compilation compared to the Baseline (template/manual approach, as set by the Raid Team). Highscore denotes the maximum number of vulnerabilities detected in the challenge. All remaining rows provide detailed metrics for the agent workflow.

Stage	Count	Proportion (%)
Baseline (template/manual, Raid Team)	1	–
Highscore (LLM Agent)	16	–
Plugins generated (LLM Agent)	16	100%
Compiled on first attempt	8	50%
Syntax errors on first attempt	8	50%
Compiled after one debugging iteration	8	100% of errors
Plugins requiring manual fixes	0	0%

The scoring for the challenge was based purely on the number of detected vulnerabilities, i.e., each detected vulnerability contributed one point to the final score. However, this metric primarily reflected how many vulnerability contexts, i.e., instructions or challenge descriptions could be supplied to the agent in advance, rather than directly measuring the agent's autonomous capabilities. As a result, our focus shifted toward maximizing the agent's ability to autonomously generate plugins that not only compiled but also required no manual review or debugging. This priority led to the development and implementation of the automated debugging loop, which ultimately resulted in the successful compilation of all generated plugins.

5 DISCUSSION

Our project demonstrates the practical application of LLMs in cybersecurity, specifically in automating the creation of vulnerability

scanner plugins. The success with the OWASP Juice Shop vulnerabilities indicates that this approach is highly viable for known vulnerabilities where a description or exploit strategy is available. A notable advantage we observed is the LLM's ability, when given a well-specified context, to creatively generate appropriate attack payloads and verification methods, mimicking the reasoning process of a human security researcher. However, this creativity is bounded by the quality and specificity of the input. Vague or underspecified descriptions can lead to incorrect or ineffective plugin logic. In our case, we mitigated this risk by providing the agent with detailed hints and structured challenge descriptions.

From an error analysis perspective, the mistakes encountered were overwhelmingly syntactic or minor logical issues. The agent rarely fundamentally misunderstood the vulnerability. This underscores the critical role of the automated debugging and correction loops, as even state-of-the-art LLMs are not infallible code generators, and reliable automation requires integrating feedback, such as compiler errors, as an explicit part of the workflow.

While our focus was on Java-based Tsunami plugins for the Juice Shop, the underlying methodology is broadly applicable. Tsunami also supports plugins written in custom YAML or Python, and, in principle, an LLM agent could generate those formats as well, perhaps even more easily. The general pattern from generation, compilation/testing to automated error-driven revision can be adapted to a wide range of security automation tasks, e.g., generating new scanner scripts for emerging CVEs. This positions LLM-powered agents as promising tools for rapidly extending and updating security tooling as new threats emerge.

Despite these promising results, our approach and evaluation setting have notable limitations. The "blindfolded" classroom challenge, in which the set of scoring vulnerabilities was undisclosed, incentivized a broad, multi-attack approach, where one plugin implemented multiple attack vectors to find the undisclosed vulnerability, rather than the tightly scoped, issue-driven process seen in real-world plugin development. In official Tsunami workflows, contributors propose and develop plugins for clearly defined vulnerabilities, subject to team review, documentation, and test cases, whereas in our setting, the score reflected not only the agent's capability, but also the volume and specificity of context supplied in advance.

Furthermore, while the challenge's final goal was to develop an official Tsunami plugin, the standards and workflows for production plugins diverged significantly from those in the Juice Shop task. Creating a real Tsunami plugin would have required developing a new, separate agent to meet stricter requirements and review processes. This highlights the need for a more streamlined and realistic progression in future challenge designs, such as specifying a defined set of vulnerabilities and then transitioning to real-world plugin development.

A few technical limitations remain. Our results benefited from using a state-of-the-art model with some implicit knowledge of common vulnerability patterns and APIs. For entirely new frameworks, substantial additional context and examples would be necessary. Furthermore, our approach primarily targeted straightforward vulnerabilities. More complex, multi-step exploits or logic flaws may require chaining multiple agents or incorporating deeper semantic analysis.

6 CONCLUSION

We have presented a case study on using an LLM-based agent to automate the development of vulnerability scanner plugins, with Google's Tsunami scanner and the OWASP Juice Shop as the primary test scenario. The agent was able to generate fully functional Java plugins for a range of web vulnerabilities, meeting the framework's requirements in the challenge environment. Key to this success was an iterative workflow: the LLM generates code and self-corrects based on compiler feedback, enabling a loop of continuous improvement. This approach eliminated the need for manual code writing or debugging, drastically accelerating the development cycle. Of the 16 tested plugins, all were generated and verified with no human in the loop.

Our results underscore the significant potential of AI in augmenting security engineering. By offloading repetitive coding tasks to an intelligent agent, security researchers can focus on higher-level analysis, such as prioritizing which vulnerabilities to target and providing the necessary context. As LLMs improve in understanding and reliability, such agents could become standard assistants in security teams, accelerating the creation of scanners, exploits, and patches.

While challenges remain in ensuring the correctness and safety of AI-generated code, and in aligning classroom challenges more closely with real-world, issue-driven development, our project demonstrates a concrete step forward. The combination of LLM reasoning with practical tool integration opens up new ways for automating and scaling the development of security solutions.

A PLUGIN GENERATION PROMPT

```
analysis_prompt =
    ChatPromptTemplate.from_messages([
    ("system", """You are an expert
        security researcher creating
        Tsunami Security Scanner plugins.
Your task is to analyze vulnerability
    information and generate a complete
    Java implementation for the
    `isServiceVulnerable` method.

WORKFLOW:
1. First, read the example SQL injection
    detector to understand the correct API
    patterns.
2. Analyze the vulnerability information
    from the `vulnerability_reader_tool`.
3. Generate the complete Java code for the
    `isServiceVulnerable` method, including
    all necessary helper methods, following
    the patterns from the example.
4. List any additional Java imports
    required by your code.

CRITICAL REQUIREMENTS:
```

```
- Your FINAL output must be only the Java
    code and the imports.
- Start with the list of imports, prefixed
    with "IMPORTS:".
- After the imports, provide the complete
    Java code for the `isServiceVulnerable`
    method, wrapped in a ```java ... ```
    markdown block.
- Do NOT include any other text,
    explanations, or formatting.
- In previous iterations you tried to
    import
    com.google.common.net.HttpHeaders;.
    This causes errors. Don't import it.
- The code must test the Application on all
    generated attacks, record the result of
    each of them and after executing the
    last attack, return #succesful attacks
    > 0

EXAMPLE OUTPUT:
IMPORTS:
import com.google.protobuf.ByteString;
import java.net.URLEncoder;

```java
private boolean
    isServiceVulnerable(NetworkService
    networkService) {{
// Complete implementation following
    example patterns
return false;
}}
```
"""),
    ("human", """Analyze the vulnerability
        details for `{vulnerability_type}`
        and generate the Java code and
        imports."""),
    ("placeholder", "{agent_scratchpad}")
])
```

B DEBUGGING PROMPT

```
fix_prompt =
    ChatPromptTemplate.from_messages([
    ("system", """You are a Java debugging
        expert. The provided Java code for
        a Tsunami plugin has a build error.
Your task is to analyze the error message
    and the code, then provide a corrected
    version of the code.

CRITICAL REQUIREMENTS:
- ONLY output the corrected, complete Java
    code for the `isServiceVulnerable`
    method.
- Do NOT output any explanations, markdown,
    or JSON.
- Ensure the corrected code is a single
    block of text.
- Pay close attention to the error message
    to identify the exact problem (e.g.,
    missing imports, syntax errors,
    incorrect method calls).
"""),
    ("human", """The following Java code
        failed to build. Please fix it.

Build Error:
{build_error}

Faulty Java Code:
{java_code}

Provide the corrected and complete
    `isServiceVulnerable` method
    implementation:
""")
])
```

REFERENCES

[1] Anthropic. 2025. Introducing Claude 4. https://www.anthropic.com/news/claude-4
[2] Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain
[3] Google. [n. d.]. Tsunami documentation. https://google.github.io/tsunami-security-scanner/howto/ [Accessed: 30-Jul-2025].
[4] Google. 2025. Gemini CLI: your open-source AI agent. https://blog.google/technology/developers/introducing-gemini-cli-open-source-ai-agent/
[5] Guoli Ma, Sebastian Lekies, Claudio Criscione, and Google Vulnerability Management Team. 2021. Announcing New Patch Reward Program for Tsunami Security Scanner. https://security.googleblog.com/2021/09/announcing-new-patch-reward-program-for.html
[6] OpenAI. 2025. Introducing Codex. https://openai.com/index/introducing-codex/
[7] OWASP. [n. d.]. Challenge solutions · Pwning OWASP Juice Shop. https://help.owasp-juice.shop/appendix/solutions.html [Accessed: 30-Jul-2025].
[8] OWASP. [n. d.]. OWASP Juice Shop | OWASP Foundation. https://owasp.org/www-project-juice-shop/ [Accessed: 30-Jul-2025].